

実験ログ A: プロシージャルモデリング 基礎

2026-09-13 | 収録 5 件

hython で実際にノードを組み、その結果を実測して記録したもの。数値はすべて実行して得た値で、推測は含まない。

各実験は「何を作ったか」「ノードの接続」「変更したパラメータ」「結果」「わかったこと」「次にやること」の順で並んでいる。用語は別ソースの「用語集: Houdini 初心者向け」にまとめてある。

収録している実験

- Box を山にして細分割し、位置で色を付ける
- subdivide の回数で、形と数はどう変わるか
- mountain と subdivide は、順序を変えると何が変わるか
- mountain のノイズパラメータは、それぞれ何に効くのか
- basis（ノイズの種類）14通りで、形はどこまで変わるか

Box を山にして細分割し、位置で色を付ける

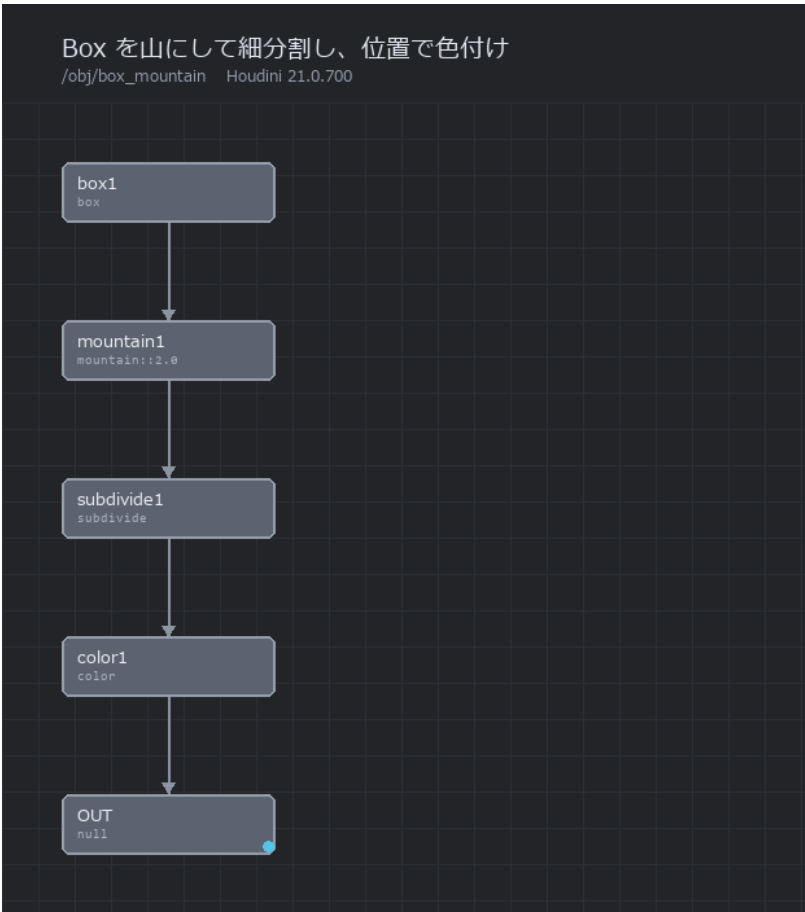
2026-09-13 | Houdini 21.0.700 | /obj/box_mountain | ノード 5 / 接続 4

何を作ったか

プロシージャルモデリングの最小構成を確認するための実験。box SOP で立方体を作り、mountain SOP でノイズによる凹凸を与え、subdivide SOP で細分割し、color SOP で色を乗せた。最後に null ノードを OUT として置き、そこを出力点にしている。

Houdini の基本的な考え方である「ジオメトリを次のノードに流し込んで加工を積み重ねる」形をそのまま辿っている。各ノードは前のノードの出力だけを入力として受け取るため、途中のどのノードでも結果を確認できる。

ノードの接続

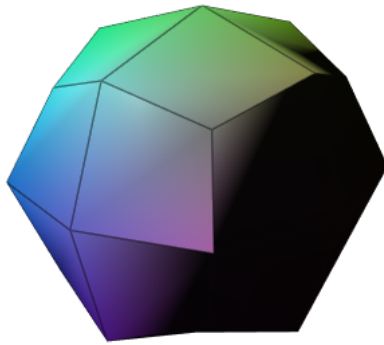


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
mountain1	mountain::2.0	elementsize = 0.8 height = 0.35
subdivide1	subdivide	(変更なし)
color1	color	colortype = 1 ramp2cb = 1.0 ramp2cg = 1.0 ramp2cr = 1.0 ramp2pos = 1.0

ノード	タイプ	デフォルトから変更したパラメータ
OUT	null	(変更なし)

結果



Houdini

OpenGL ROP による Smooth Wire Shaded

表示。ワイヤーフレームが乗っているため、ネットワークが生成したトポロジが確認できる。画像内のウォーターマークは Apprentice ライセンスによるもの。

ジオメトリ

ポイント数	26
プリミティブ数	24
バウンディングボックス	[-0.4794, -0.4768, -0.4767] ~ [0.4932, 0.4959, 0.4959]
ポイント属性	P, Cd
プリミティブ属性	なし
ディテール属性	なし

わかったこと

- ・ box は 8 ポイント / 6 プリミティブ。subdivide を 1 回かけると 26 ポイント / 24 プリミティブになる。Catmull-Clark 細分割なので、回数を増やすと立方体はどんどん球に近づく（2 回で既に球状になり、元が箱だと分からなくなった）。

- ・ mountain SOP

は既存のポイントを法線方向にノイズで押し出すもので、ポイントを増やさない。つまり box のままかけても 8 点しか動かないため、凹凸のディテールを出すには先に subdivide などで分割数を上げる必要がある。

- ・ color SOP の colortype は数値メニュー（0=Constant, 1=Bounding Box, 2=Random, 3=Ramp from Attribute, 4=Random from Attribute）。文字列で指定すると TypeError になる。今回は 1

を使い、バウンディングボックス内の位置をそのまま RGB に割り当てた。

- ・ color SOP を通すとポイント属性に Cd が追加される（P のみ → P, Cd）。属性が増えたかどうかはジオメトリの統計で確認できる。

次にやること

- ・ subdivide の回数を変えて、ポイント数と見た目がどう変わるかを並べて比較する
- ・ mountain を subdivide の後ろに移動し、分割後に凹凸をかけた場合との違いを見る
- ・ grid から地形を作る流れに広げ、プロシージャルモデリングの定番パターンに入る

subdivide の回数で、形と数はどう変わるか

2026-09-13 | Houdini 21.0.700 | /obj/subdiv_compare | ノード 5 / 接続 4

何を作ったか

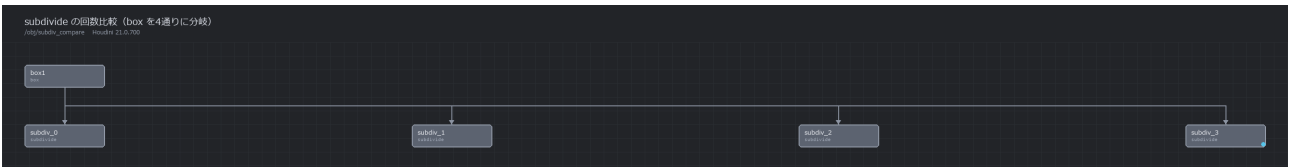
実験001で「subdivideの回数を増やすと立方体が球に近づく」ことに気づいたので、それを数値で確かめる実験。box SOP に subdivide SOP

を4通り（0回・1回・2回・3回）ぶら下げ、すべて同じカメラでレンダして並べた。

同じカメラで撮っているのが重要な点で、各変種を自動でフレームに収めてしまうと、大きさの違いが消えて比較にならない。画を見ると球が立方体よりはっきり小さいことが分かるが、これは目の錯覚ではなく実測でも縮んでいる。

さらに「なぜ縮むのか」「どこまで縮むのか」「1回目だけ縮まないのはなぜか」を追加検証で確定させた。

ノードの接続

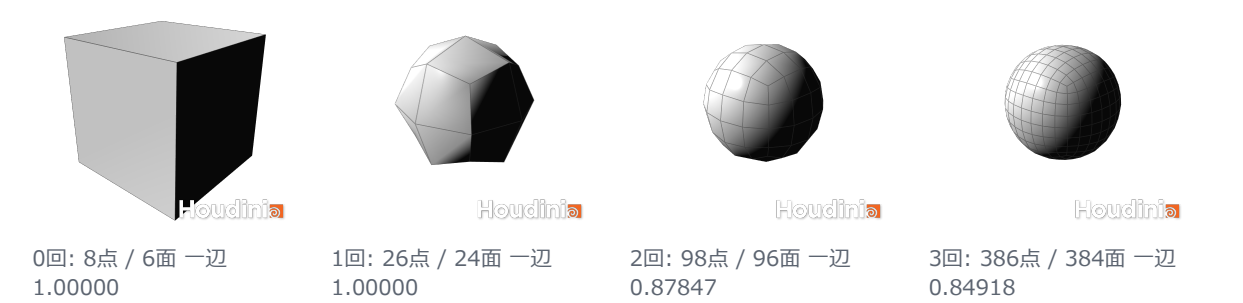


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
subdiv_0	subdivide	iterations = 0
subdiv_1	subdivide	(変更なし)
subdiv_2	subdivide	iterations = 2
subdiv_3	subdivide	iterations = 3

回数を変えた結果

すべて同じカメラ・同じ表示モード（Smooth Wire Shaded）。元の立方体の輪郭に対して、分割を重ねるほど内側に縮んでいくのが分かる。



iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
0	8	6	—	1.00000
1	26	24	4.00	1.00000

iterations	ポイント数	プリミティブ数	面数の倍率	一辺の長さ
2	98	96	4.00	0.87847
3	386	384	4.00	0.84918
4	1538	1536	4.00	0.84192
5	6146	6144	4.00	0.84011
6	24578	24576	4.00	0.83966

アルゴリズム別の比較（iterations = 3 で固定）

subdivide SOP の algorithm パラメータには5つの選択肢がある。既定値は 2（OpenSubdiv Catmull-Clark）。同じ3回分割で結果がどう違うかを測った。

algorithm	ポイント数	プリミティブ数	一辺の長さ
Houdini Catmull-Clark	386	384	0.84918
Mantra-Compatible Catmull-Clark	386	384	0.83951
OpenSubdiv Catmull-Clark（既定）	386	384	0.84918
OpenSubdiv Loop	386	768	0.86932
OpenSubdiv Bilinear	386	384	1.00000

回数を変えたときの一辺（2つのアルゴリズムの比較）

OpenSubdiv Catmull-Clark は回数を増やすほど値が動くが、Mantra-Compatible は回数を変えても一定。これが何を意味するかは「わかったこと」の最後に書いた。

iterations	OpenSubdiv CC	Mantra-Compatible
1	1.00000	0.83951
2	0.87847	0.83951
3	0.84918	0.83951
4	0.84192	0.83951
5	0.84011	0.83951

わかったこと

- iterations = 0 は素通し。8点 / 6面のまま何も起きない。比較の基準として使える。
- プリミティブ数は1回ごとに正確に4倍になる（6 → 24 → 96 → 384 → 1536 → 6144 → 24576）。四角形1枚が4枚に割れるため。3回で64倍、6回で4096倍なので、回数を安易に増やすと破綻する。
- ポイント数は常に「プリミティブ数 + 2」になっている。これは偶然ではなくオイラーの多面体定理から導ける。閉じた四角形メッシュでは各辺を2つの面が共有するので $E = 2F$ 、これを $V - E + F = 2$ に代入すると $V = F + 2$ 。実測した7通りすべてで一致した。
- 1回目だけ一辺が 1.00000 のまま縮まない。検証したところ、+X方向の端を保持しているポイントは (0.5, 0, 0) の1点だけだった。これは元の +X 面の中心そのもの。Catmull-Clark は各面の中心に新しい点を作るため、1回目はその新点元が元の面の上に乗り、バウンディングボックスが変わらない。2回目でその点も内側に引かれ 0.87847 に縮む。

- ・ 一辺の縮み方は等比的に収束する。差を取ると $0.02929 \rightarrow 0.00726 \rightarrow 0.00181 \rightarrow 0.00045$ で、毎回およそ $1/4$ （実測 $0.248 / 0.249 / 0.249$ ）。ここから外挿した極限值は約 0.83951 になる。

- ・ 縮みの原因は「分割」ではなく「平滑化」だと確定した。OpenSubdiv Bilinear（平滑化せず分割するだけの方式）で3回分割しても一辺は 1.00000 のまま変わらない。面数は同じ384なので、分割量は同じで平滑化の有無だけが違う。

- ・ OpenSubdiv Loop だけプリミティブ数が 768 になる（他は384）。Loop は三角形メッシュ向けの方式なので、四角形が三角形に割られて枚数が倍になる。四角形を保ちたい場面では選んではいけない。

- ・ Mantra-Compatible Catmull-Clark は回数を $1 \sim 5$ のどれにしても一辺が 0.83951 で一定だった。これは分割を繰り返して極限に近づけるのではなく、極限曲面上の位置を直接求めていることを示している。実際この値は、OpenSubdiv CC の収束列から外挿した極限值 0.83951 と一致した。つまり「無限回分割した先の形」を1回で出しているのがこのモード。

次にやること

- ・ crease（折り目）を使って角を保持できるか。立方体らしさを残したまま分割する方法があるはず

- ・ mountain と subdivide

の順序を入れ替えて、凹凸のディテールがどう変わるかを比較する（実験003）

- ・ polyextrude

など他のSOPでもポイント数・プリミティブ数の増え方を測り、重くなる原因を数で把握する

mountain と subdivide

は、順序を変えると何が変わるか

2026-09-13 | Houdini 21.0.700 | /obj/order_compare | ノード 14 / 接続 13

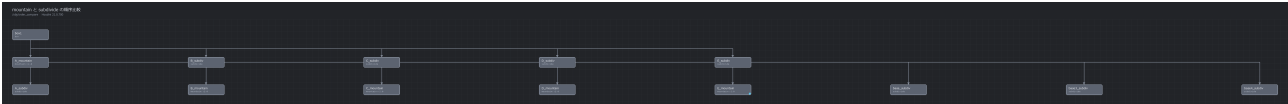
何を作ったか

実験001で「mountain はポイントを増やさないなので、細かい凹凸を出すには先に分割数を上げる必要がある」と書いた。これは筋は通っているが、実際に確かめてはいなかった。そこで順序違いを並べて比較し、違いを数字で出す。

比較したのは6通り。ノイズなしの基準形、ノイズを先にかけたもの、分割を先にしたもの、分割をさらに細かくしたもの、ノイズの特徴サイズを小さくしたもの、そのうえで点数を増やしたもの。すべて同じカメラでレンダした。

この実験では測り方を一度間違えている。最初に選んだ指標では違いが正しく出ず、途中で別の指標に切り替えた。その経緯も含めて記録した。

ノードの接続

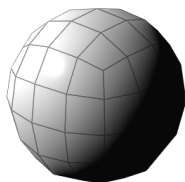


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
A_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
A_subdiv	subdivide	iterations = 2
B_subdiv	subdivide	iterations = 2
B_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
C_subdiv	subdivide	iterations = 3
C_mountain	mountain::2.0	elementsiz = 0.8 height = 0.35
D_subdiv	subdivide	iterations = 3
D_mountain	mountain::2.0	elementsiz = 0.25 height = 0.35
E_subdiv	subdivide	iterations = 4
E_mountain	mountain::2.0	elementsiz = 0.25 height = 0.35
base_subdiv	subdivide	iterations = 2
base3_subdiv	subdivide	iterations = 3
base4_subdiv	subdivide	iterations = 4

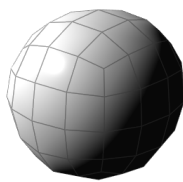
6通りの結果

すべて同じカメラ・同じ表示モード。左から右へ、凹凸がどう変わるかを見てほしい。mountainのパラメータは height = 0.35 で固定し、elementsize（ノイズの特徴サイズ）だけを D と E で 0.8 から 0.25 に変えている。



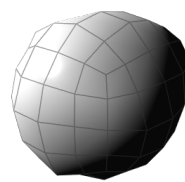
Houdini

基準: 分割のみ、ノイズなし 98点 / 96面



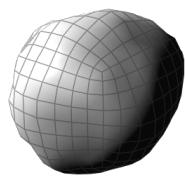
Houdini

A: mountain → subdiv(2) 98点 / 96面



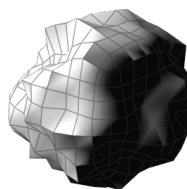
Houdini

B: subdiv(2) → mountain 98点 / 96面



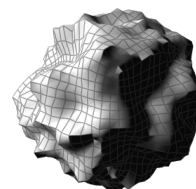
Houdini

C: subdiv(3) → mountain 386点 / 384面



Houdini

D: C と同じだが elementsize 0.25 / 386点



Houdini

E: D と同じノイズで 点数のみ増加 / 1538点

基準形からの点ごとのずれ

同じ分割回数ならポイントの順序も同じなので、ノイズなしの基準形と1点ずつ引き算できる。これが「ノイズがどれだけ形を動かしたか」を直接表す。A・B は2回分割の基準形と、C・D は3回分割の基準形と、E は4回分割の基準形と比べている。

変種	ポイント数	平均のずれ	最大のずれ
A: mountain → subdiv(2)	98	0.02069	0.03671
B: subdiv(2) → mountain	98	0.02479	0.08062
C: subdiv(3) → mountain	386	0.02504	0.08009
D: C + elementsize 0.25	386	0.03236	0.11985
E: D + 点数4倍	1538	0.03239	0.13173

最初に選んで失敗した指標（表面積）

凹凸が増えれば表面積も増えるはずだと考え、まず表面積を測った。ところが mountain を足すと表面積は減った。ノイズは外側にも内側にも押すため全体が縮み（一辺が 0.8785 から 0.8545

へ)、面積は大きさの2乗に比例するのでその影響が凹凸の効果を打ち消していた。一辺の2乗で割って正規化しても、値の大小が凹凸の量と対応しない。大きさと凹凸が混ざった指標だったということ。

変種	表面積	基準比	一辺	面積 / 一辺 ²
基準 (ノイズなし)	2.43164	1.000	0.8785	3.15076
A	2.30052	0.946	0.8545	3.15066
B	2.24947	0.925	0.8535	3.08797
C	2.16988	0.892	0.8314	3.13918
D	2.49922	1.028	0.9123	3.00282
E	2.66892	1.098	0.9278	3.10046

わかったこと

- ・順序を入れ替えてもポイント数とプリミティブ数は同じ (どちらも98点 / 96面)。つまりデータの重さは変わらない。変わるのは形だけ。
- ・最大のずれは A が 0.03671、B が 0.08062 で、B が約2.2倍大きい。同じ mountain の設定でも、分割したあとにかけた方が凹凸が残る。実験001で立てた見込みは正しかった。
- ・なぜそうなるかも測れた。box に mountain をかけた直後は、8点すべてが平均 0.03607 動いている。ところがそのあと subdivide をかけると、基準形からのずれは平均 0.02069・最大 0.03671 まで下がる。Catmull-Clark の平滑化は周囲の点を平均するので、せっかく動かした凹凸をなめしてしまう。
- ・C は B とほぼ同じずれだった (平均 0.02504 対 0.02479、最大 0.08009 対 0.08062)。点を4倍に増やしても凹凸の大きさは変わらない。点数はノイズの振幅を決めていない。
- ・振幅を決めていたのは elementsize だった。C と D は同じ386点だが、elementsize を 0.8 から 0.25 に下げると平均のずれが 0.02504 から 0.03236 に増え、画も明確に岩ようになった。
- ・E は D と同じノイズ設定で点数だけ4倍にしたもの。平均のずれは 0.03236 から 0.03239 でほぼ変わらない。一方で画は、同じ起伏をより滑らかな面で表現できている。つまり点数は「ノイズをどれだけ忠実に表現できるか (解像度)」を決めていて、「起伏の大きさ (振幅)」は決めていない。
- ・ただし最大のずれだけは点数とともに少し増える (0.11985 → 0.13173)。点が増えるとノイズのより極端な場所まで拾えるようになるため、振幅そのものが増えたわけではない。
- ・実用的な結論: 細かいディテールが欲しいときは、分割数を上げるだけでは足りない。elementsize も一緒に小さくする。そして mountain は分割のあとに置く。この3つが揃って初めて狙った凹凸になる。
- ・測り方についての反省。表面積のような「全体をひとつの数にまとめた量」は、大きさと形の変化が混ざってしまう。基準形と1点ずつ対応づけて引き算する方が、知りたいことを素直に測れた。比較実験では、まず基準形を作り、それと1対1で比べられる量を選ぶべき。

次にやること

- ・mountain の他のパラメータ (roughness, octaves) が振幅と細かさのどちらに効くのかを、同じやり方で切り分ける
- ・grid から地形を作り、平面の場合も同じ結論になるか確かめる (実験004)
- ・crease を使って、分割しても角を保持できるかを確かめる (実験002からの持ち越し)

mountain

のノイズパラメータは、それぞれ何に効くのか

2026-09-13 | Houdini 21.0.700 | /obj/mountain_params | ノード 15 / 接続 14

何を作ったか

実験003で「点数は解像度を決め、elementsize は振幅を決める」と分かった。では残りのノイズパラメータは何を変えているのか。同じ切り分け方で調べる。

最初に分かったのは名前の問題だった。パラメータ名は roughness や octaves ではなく rough / oct / lac。実際に mountain::2.0 が持っている名前を確認してから使わないと、そもそも動かない。

この実験でも測り方を追加している。003で使った「基準形からのずれ」だけでは、これらのパラメータをほとんど区別できなかった。振幅ではなく形の性質を変えるパラメータだったため、性質を測る指標を新しく用意した。

ノードの接続



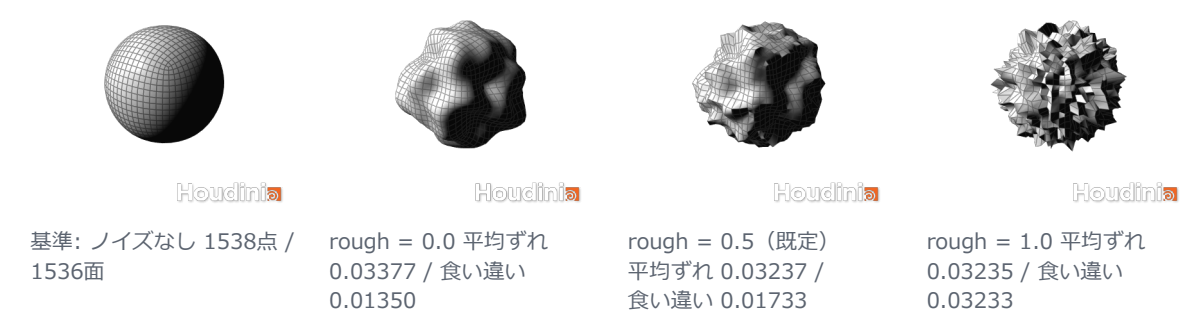
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
rough_00	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 0.0
rough_05	mountain::2.0	elementsize = 0.25 height = 0.35
rough_10	mountain::2.0	elementsize = 0.25 height = 0.35 rough = 1.0
oct_1	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 1
oct_3	mountain::2.0	elementsize = 0.25 height = 0.35 oct = 3
oct_8	mountain::2.0	elementsize = 0.25 height = 0.35
lac_15	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 1.5
lac_20	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 2.0
lac_40	mountain::2.0	elementsize = 0.25 height = 0.35 lac = 4.0

ノード	タイプ	デフォルトから変更したパラメータ
fractal_hmFT	mountain::2.0	elementsiz = 0.25 height = 0.35
fractal_fBm	mountain::2.0	elementsiz = 0.25 fractal = fBm height = 0.35
fractal_mfT	mountain::2.0	elementsiz = 0.25 fractal = mfT height = 0.35
fractal_none	mountain::2.0	elementsiz = 0.25 fractal = none height = 0.35

rough（粗さ） — 振幅は変えず、細かさだけを変える

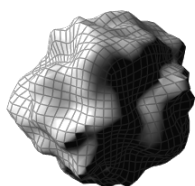
固定条件はすべての変種で共通: subdivide 4回（1538点）、height 0.35、elementsiz 0.25。rough だけを動かしている。平均のずれはほぼ変わらないのに、見た目はなだらかな丘から棘だらけの形まで変わる。



rough	平均のずれ	最大のずれ	隣との食い違い
0.0	0.03377	0.14799	0.01350
0.5（既定）	0.03237	0.13153	0.01733
1.0	0.03235	0.13588	0.03233

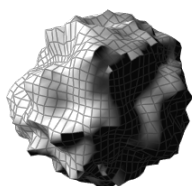
oct（オクタブ数） — 3 以上で効かなくなる

オクタブとは、細かさの違うノイズを重ねる層のこと。oct はその層の数。1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733）。点数が足りず、細かい層を表現できていないため。



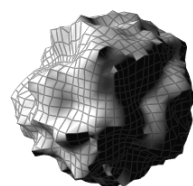
Houdini

oct = 1 食い違い 0.01519



Houdini

oct = 3 食い違い 0.01732



Houdini

oct = 8 (既定) 食い違い 0.01733

oct	平均のずれ	最大のずれ	隣との食い違い
1	0.03231	0.13417	0.01519
3	0.03250	0.13274	0.01732
8 (既定)	0.03237	0.13153	0.01733

予測の検証 — oct が効かないのは点数のせいかな

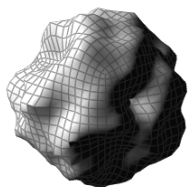
「oct が 3 以上で効かないのは点数が足りないからだ」という説明が正しいなら、点数を増やせば oct=8 が oct=3 を上回るはず。実際にそうだった。386点では oct=8 が oct=3 を下回っているが、6146点では上回っている。

注意: 食い違いの絶対値は点数が増えると小さくなる（隣同士の距離が縮まるので差も縮む）。この表は各行の中で比較だけが有効で、行をまたいで比べてはいけない。

subdivide	点数	oct = 1	oct = 3	oct = 8	8 と 3 の差
3回	386	0.02692	0.02868	0.02856	-0.00012
4回	1538	0.01519	0.01732	0.01733	+0.00001
5回	6146	0.00782	0.00941	0.00963	+0.00022

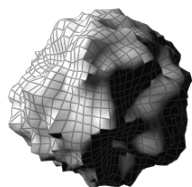
lac（ラキュナリティ） — オクターブ間の周波数の開き

重ねる層ごとに、どれだけ細かさを飛ばすかの倍率。既定は 1.841。大きくすると層の間隔が開いて細かい成分が強くなる。ただし lac = 1.5 では平均のずれも下がっており、振幅にも影響が出ている。



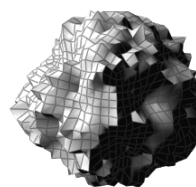
Houdini

lac = 1.5 平均 0.02625 / 食い違い 0.01259



Houdini

lac = 2.0 平均 0.03160 / 食い違い 0.01734



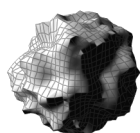
Houdini

lac = 4.0 平均 0.03169 / 食い違い 0.02218

lac	平均のずれ	最大のずれ	隣との食い違い
1.5	0.02625	0.13925	0.01259
2.0	0.03160	0.13528	0.01734
4.0	0.03169	0.13334	0.02218

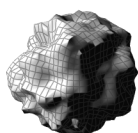
fractal（フラクタルの種類） — 差は小さい

層の重ね方の方式。4種類あるが、今回の条件では数値の差が小さかった。Terrain だけやや滑らか。既定は Hybrid Terrain。



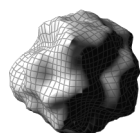
Houdini

Hybrid Terrain（既定）
食い違い 0.01733



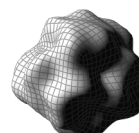
Houdini

Standard (fBm) 食い違い
0.01786



Houdini

Terrain 食い違い 0.01481



Houdini

None 食い違い 0.01350

fractal	平均のずれ	最大のずれ	隣との食い違い
Hybrid Terrain（既定）	0.03237	0.13153	0.01733
Standard (fBm)	0.03183	0.13207	0.01786
Terrain	0.03288	0.13504	0.01481
None	0.03377	0.14799	0.01350

わかったこと

- ・パラメータ名は rough / oct / lac。roughness や octaves という名前は存在せず、そのまま書くと何も設定されない。既定値は rough = 0.5、oct = 8、lac = 1.841、fractal = Hybrid Terrain、basis = Simplex。使う前に実際の名前を確認する必要がある。
- ・003で使った「基準形からのずれ」だけでは、これらのパラメータはほとんど区別できなかった。平均のずれはどれも 0.032~0.034 の範囲に収まる。振幅を決めているのは height と elementsize であり、今回のパラメータは振幅に効いていない。
- ・そこで「隣り合う点どうして、ずれの向きと量がどれだけ食い違うか」を測る指標を足した。なだらかな起伏なら隣の点も同じように動くので値は小さく、細かい凹凸なら大きくなる。基準形は原点中心なので、中心からの距離の差を符号付きのずれとして使っている（外に出たら正、へこんだら負）。これで完全に分離できた。
- ・rough が細かさに最も強く効く。0.0 から 1.0 で食い違いが 0.01350 から 0.03233 へ約2.4倍になる一方、平均のずれは 0.03377 から 0.03235 でほぼ変わらない。つまり rough は、同じ振幅のまま、なだらかな丘から棘だらけの形まで性質だけを変えるパラメータ。
- ・rough = 0.0 の数値は fractal = None と完全に一致した（平均 0.03377、最大 0.14799、食い違い 0.01350）。rough = 0 では追加の層が何も足さないの、層を重ねない単一ノイズに退化している。別の経路で同じ結果に到達したので、この解釈は確かだと言える。
- ・oct は 1 から 3 では効くが、3 から 8 ではほぼ変わらない（0.01732 → 0.01733、差は 0.00001）。点数が足りず細かい層を表現できていないため。この説明を予測に変えて検証したところ、点数を増やす

と oct = 8 が oct = 3 を上回り始めた（386点では -0.00012、1538点で +0.00001、6146点で +0.00022）。予測どおりだったので、この解釈も確かだと言える。

- ・実用的には、oct を上げるだけでは無駄になる。点数が足りていなければ、上げた層は形に現れない。003の「点数は解像度を決める」という結論と同じことが、別の角度から確認できた。

- ・lac も細かさに効く（1.5 → 4.0 で 0.01259 → 0.02218）。ただし lac = 1.5 では平均のずれも 0.02625 に下がっており、振幅にも影響している。rough のようにきれいに性質だけを変えるパラメータではない。

- ・fractal の4種類は、今回の条件では差が小さかった（食い違い 0.01350~0.01786）。地形向け的方式が揃っているので、平面の地形（実験005の予定）ではもっと差が出る可能性がある。

- ・指標の注意点。食い違いの絶対値は点数が増えると小さくなる。隣同士の距離が縮まれば、その間のずれの差も縮むため。したがって同じ点数の中でだけ比較できる。点数をまたいだ比較には、辺の長さで割るなどの正規化が必要になる。

- ・basis（ノイズの基礎となる種類）は14種類あった。既定は Simplex。Worley Cellular のようなセル状のものも含まれており、見た目が大きく変わるはずなので、別の実験にする価値がある。

次にやること

- ・basis を14種類すべて描き出して並べる。セル状ノイズは岩や結晶の見た目に直結するので、実用価値が高い
- ・grid から地形を作り、平面では fractal の種類の差が出るかを確かめる（実験005）
- ・食い違いの指標を辺の長さで正規化し、点数をまたいだ比較ができるようにする

basis（ノイズの種類） 14通りで、形はどこまで変わるか

2026-09-13 | Houdini 21.0.700 | /obj/noise_basis | ノード 16 / 接続 15

何を作ったか

004で basis に14種類あることが分かったので、同じ条件で全部描き出した。height 0.35、elementsize 0.25、subdivide 4回（1538点）で固定し、basis だけを変えている。

結果は他のパラメータと性質がまったく違った。rough は振幅を変えずに形の性質だけを変えたが、basis は振幅も性質も両方変える。同じ height を指定しても、種類によって実際のずれが4.5倍違う。

測る指標には004で作った「隣との食い違い」に加えて、外に出た点の割合を足した。ノイズが外向きに押すのか内向きに削るのかで、使いどころが変わるため。

ノードの接続



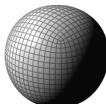
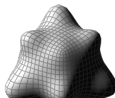
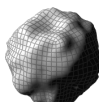
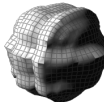
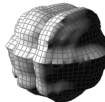
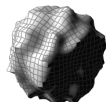
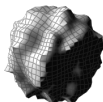
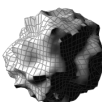
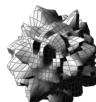
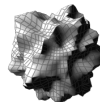
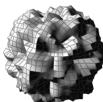
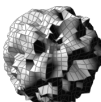
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
box1	box	(変更なし)
base_subdiv	subdivide	iterations = 4
basis_sine	mountain::2.0	basis = sine elementsize = 0.25 height = 0.35
basis_perlin	mountain::2.0	basis = perlin elementsize = 0.25 height = 0.35
basis_pperlin	mountain::2.0	basis = pperlin elementsize = 0.25 height = 0.35
basis_simplex	mountain::2.0	elementsize = 0.25 height = 0.35
basis_sparse	mountain::2.0	basis = sparse elementsize = 0.25 height = 0.35
basis_flow	mountain::2.0	basis = flow elementsize = 0.25 height = 0.35
basis_pflow	mountain::2.0	basis = pflow elementsize = 0.25 height = 0.35
basis_worleyFA	mountain::2.0	basis = worleyFA elementsize = 0.25 height = 0.35
basis_worleyFB	mountain::2.0	basis = worleyFB elementsize = 0.25 height = 0.35

ノード	タイプ	デフォルトから変更したパラメータ
basis_mworleyFA	mountain::2.0	basis = mworleyFA elementsize = 0.25 height = 0.35
basis_mworleyFB	mountain::2.0	basis = mworleyFB elementsize = 0.25 height = 0.35
basis_cworleyFA	mountain::2.0	basis = cworleyFA elementsize = 0.25 height = 0.35
basis_cworleyFB	mountain::2.0	basis = cworleyFB elementsize = 0.25 height = 0.35
basis_alligator	mountain::2.0	basis = alligator elementsize = 0.25 height = 0.35

14種類すべて（細かさの小さい順）

すべて同じカメラ・同じ設定。左上のなめらかなものから、右下の硬く割れたものへ並んでいる。

					
Houdini	Houdini	Houdini	Houdini	Houdini	
基準: ノイズなし 1538点	Sinusoid 平均 0.14454 / 細かさ 0.00444	Sparse Convolution 平均 0.01985 / 細かさ 0.00965	Periodic Perlin Flow 平均 0.01551 / 細かさ 0.01005	Periodic Perlin 平均 0.01662 / 細かさ 0.01062	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Perlin 平均 0.02426 / 細かさ 0.01268	Perlin Flow 平均 0.02639 / 細かさ 0.01441	Simplex (既定) 平均 0.03237 / 細かさ 0.01733	Worley F2-F1 平均 0.14144 / 細かさ 0.01831	Worley F1 平均 0.08017 / 細かさ 0.02065	
					
Houdini	Houdini	Houdini	Houdini	Houdini	
Alligator 平均 0.12904 / 細かさ 0.02747	Manhattan F1 平均 0.08103 / 細かさ 0.02996	Chebyshev F2-F1 平均 0.11841 / 細かさ 0.03104	Chebyshev F1 平均 0.09937 / 細かさ 0.03193	Manhattan F2-F1 平均 0.11505 / 細かさ 0.03323	
basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sine	Sinusoid	0.14454	0.15748	0.00444	0.0%

basis	表示名	平均のずれ	最大のずれ	隣との食い違い	外向きの点の割合
sparse	Sparse Convolution	0.01985	0.08378	0.00965	32.9%
pflow	Periodic Perlin Flow	0.01551	0.05039	0.01005	51.2%
pperlin	Periodic Perlin	0.01662	0.05705	0.01062	58.8%
perlin	Perlin	0.02426	0.09921	0.01268	40.9%
flow	Perlin Flow	0.02639	0.10496	0.01441	49.9%
simplex	Simplex (既定)	0.03237	0.13153	0.01733	35.8%
worleyFB	Worley Cellular F2-F1	0.14144	0.17436	0.01831	0.0%
worleyFA	Worley Cellular F1	0.08017	0.17328	0.02065	5.4%
alligator	Alligator	0.12904	0.17456	0.02747	2.3%
mworleyFA	Manhattan Cellular F1	0.08103	0.16473	0.02996	91.1%
cworleyFB	Chebyshev Cellular F2-F1	0.11841	0.17439	0.03104	1.4%
cworleyFA	Chebyshev Cellular F1	0.09937	0.16540	0.03193	94.4%
mworleyFB	Manhattan Cellular F2-F1	0.11505	0.17052	0.03323	2.5%

わかったこと

- basis は振幅と性質の両方を変える。rough

が振幅を保ったまま性質だけを変えたのとは性格が違う。同じ height 0.35 でも、平均のずれは Periodic Perlin Flow の 0.01551 から Sinusoid の 0.14454 まで約9.3倍の開きがある。basis を変えたら height も再調整が必要。

- Sinusoid は例外的な挙動をする。平均のずれが 0.14454 と最大級に大きいのに、隣との食い違いは 0.00444 で全種類中もっとも小さい。外向きの点は 0.0%、つまり全部の点が内側に動いている。画を見ると、丸い4つの膨らみを持つ滑らかな形になっていた。規則的な波なので、細かい凹凸ではなく大きなうねりを作る。地形の大きな起伏には使えるが、ディテールには使えない。

- Perlin 系 (perlin / pperlin / flow / pflow) は外向きの点が 40~59% で、外と内にほぼ均等に動く。平均のずれも 0.0155~0.0264 と控えめ。自然な揺らぎを足したいときの素直な選択肢。

- 既定の Simplex は Perlin 系より振幅も細かさも大きい (平均 0.03237、細かさ 0.01733) が、セル系ほど極端ではない。中間的な位置にある。

- セル系 (Worley / Manhattan / Chebyshev / Alligator) は平均のずれが 0.08~0.14 と大きく、隣との食い違いも上位を占める。岩、割れ、結晶のような硬い見た目はここから作る。

- セル系の中で、外向きの点の割合が極端に分かれた。Manhattan F1 が 91.1%、Chebyshev F1 が 94.4% で、ほぼ全部の点が外に出る。一方 Worley F1 は 5.4%、Alligator は 2.3%、Chebyshev F2-F1 は 1.4% で、ほぼ全部が内側に削られる。

- ・画で確認すると理由が見えた。Worley F1

は、細胞の中心から遠いところが深く削られた侵食岩のような形になる。Manhattan F1 は距離の測り方が軸に沿っているため、軸に平行な面を持つ角ばった塊が外に突き出す形になった。同じ「F1」でも距離の定義が違えば、削る方向と形がまったく変わる。

・ F1 と F2-F1 の違いも外向き率に出る。Manhattan は F1 が 91.1% (外向き) なのに F2-F1 は 2.5% (内向き)。Chebyshev も F1 が 94.4%、F2-F1 が 1.4% で同じ傾向。F2-F1 は細胞の境界線を強調する計算なので、境界以外が下がる = 内向きになる。

・最大のずれはセル系でほぼ頭打ちになっている (0.164~0.175)。height 0.35 に対して 0.5 倍弱で止まっており、何らかの上限が効いている可能性がある。この確認は別の実験にする。

・外向きの点の割合という指標は安く作れて情報量が多かった。ノイズが押すのか削るのかは、実際に作りたいもの (突起か窪みか) に直結する。

次にやること

- ・セル系で最大のずれが 0.175 付近で止まる理由を確かめる。height を上げて止まるのかを測る
- ・grid の平面で同じ14種類を並べる。地形として見たときの向き不向きは球体とは違うはず
- ・外向き率を、突起を作りたいとき／窪みを作りたいときの選択指針としてまとめる